



Time Series Normalization with tspredit



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>



Normalization Examples in Time Series

A didactic progression of normalization strategies for forecasting models using the `tspredict` package in R. Each technique emphasizes different aspects of the series: shape, absolute level, or adaptation to changes over time. The adaptive family offers 4 distinct operators: **divide**, **subtract**, **softdivide**, and **asinh**.

01

Baseline

No normalization — reference for comparison
`ts_norm_none`

02

Global Scaling

Global Min-Max over the entire training set
`ts_norm_gminmax`

03

Local Scaling

Min-Max per sliding window
`ts_norm_swminmax`

04

Differencing

Replacement by relative variations `ts_norm_diff`

05

Adaptive Divisive

Division by adaptive reference level `ts_norm_an`

06

Adaptive Subtractive

Subtraction of adaptive level `ts_norm_an`,
`subtract`

07

Adaptive Softdivide

Compromise between addition and division `ts_norm_an`, `softdivide`

08

Adaptive Asinh

Smooth non-linear transformation `ts_norm_an`, `asinh`

Standard Workflow

```
# Load packages
library(daltoolbox)
library(tspredit)

# Load example series
data(tsd)

# Create sliding windows
sw_size <- 10
ts <- ts_data(tsd$y, sw_size)
```

Step-by-step overview

- Load daltoolbox and tspredit
- Load the tsd series with data(tsd)
- Organize into windows of size 10 with ts_data()

 **Note:** This setup is shared by all the following examples. Each normalization technique starts from this point and applies its own transformation.

No Normalization (`ts_norm_none`)

The absence of normalization preserves the original scale of the series. It serves as a **reference baseline**: any subsequent technique must outperform this case to justify its use. The `ts_norm_none()` operator applies an identity transformation — the output is identical to the input.

```
# (continued from Standard Workflow)
```

```
# Apply null "normalization"
```

```
preproc <- ts_norm_none()
```

```
preproc <- fit(preproc, ts)
```

```
tst <- transform(preproc, ts)
```

```
# Verify: output == input
```

```
ts_head(tst, 3)
```

```
summary(tst[,10])
```

```
# Min. :-0.99929 Max. : 0.99460
```

Step by step

- `ts_norm_none()` does not alter values — output is identical to input
- Serves as a **reference** to evaluate the impact of other techniques

📌 **Key concept:** The absence of transformation preserves the original scale. Use as a baseline to judge whether any subsequent normalization improves the predictor.

Global Min-Max Normalization (`ts_norm_gminmax`)

Global min-max normalization rescales the entire training series to the interval `[0, 1]` using the **global** minimum and maximum. It is simple and effective when the series distribution is stable over time. The relative order of values is preserved, making magnitudes comparable across different series.

```
# (continued from Standard Workflow)
# Global min-max normalization
preproc <- ts_norm_gminmax()
preproc <- fit(preproc, ts)
tst <- transform(preproc, ts)
```

```
ts_head(tst, 3)
# t9    t8    t0
# 0.5004502 0.6243512 0.8901126
# 0.6243512 0.7405486 0.8001676
```

```
summary(tst[,10])
# Min. :0.0000  Max. :0.9985
# Mean :0.5154
```

Step by step

- Fit the normalizer with `fit()` on the entire training set
- `ts_norm_gminmax()` computes the **global** minimum and maximum of the series
- All values are rescaled to the interval `[0, 1]`
- The relative order of values is preserved
- Magnitudes become comparable, facilitating learning algorithms

 **Key concept:** The global scale uses fixed statistics from training. It is simple and effective when the series distribution does not change over time.

Sliding Window Min-Max Normalization (`ts_norm_swminmax`)

Sliding window min-max normalization applies `[0, 1]` rescaling **locally** to each window, using that window's own minimum and maximum. This makes the model focus on the **shape** of the pattern rather than the absolute level — useful when the amplitude of the series varies over time.

```
# (continued from Standard Workflow)
```

```
# Local window normalization
```

```
preproc <- ts_norm_swminmax()
```

```
preproc <- fit(preproc, ts)
```

```
tst <- transform(preproc, ts)
```

```
ts_head(tst, 3)
```

```
# Window 1: min=0, max=1 (local)
```

```
# Window 2: min=0, max=1 (local)
```

```
summary(tst[,10])
```

```
# Min. :0.0000 Max. :1.0000
```

```
# Median :0.2264 Mean :0.4301
```

Step by step

- Each window is normalized using its **own** local minimum and maximum
- The model always receives values in `[0, 1]` regardless of amplitude
- Emphasizes the **shape** of the series rather than the absolute level
- Useful when amplitude varies over time
- Compare with global: level information is removed here

📌 **Key concept:** Local scaling focuses on the pattern within each window. It loses absolute level information, but gains robustness to amplitude variations.

Adaptive Normalization (ts_norm_an)

Divisive adaptive normalization (operation = "divide", default) rescales each window by its **adaptive reference level**, estimated over the full supervised window. Unlike global scaling, the reference level evolves over time — suitable for non-stationary series with level drift.

```
# (continues from Standard Workflow)
```

```
# Adaptive normalization
```

```
preproc <- ts_norm_an()
```

```
preproc <- fit(preproc, ts)
```

```
tst <- transform(preproc, ts)
```

```
ts_head(tst, 3)
```

```
# t0: 0.5424, 0.4300, 0.3218
```

```
summary(tst[,10])
```

```
# Min. :0.04995 Max. :0.94995
```

```
# Median :0.41183 Mean :0.44854
```

Step by step

- ts_norm_an() updates the scaling statistics **over time**
- Does not use a fixed global range — tracks the series drift
- Suitable when the distribution changes (non-stationary series)
- The predictor follows the current scale instead of a historical reference

📌 **Key concept:** Normalization as a time-varying transformation. Reference: Ogasawara et al. (2010), IJCNN — *Adaptive Normalization for non-stationary time series*.

Adaptive Subtractive Normalization (`ts_norm_an – subtract`)

The subtractive variant removes the **adaptive reference level** from each window by subtraction, instead of division. It is the safer choice when the series may have values close to zero, where division becomes numerically unstable. The result is interpreted as an additive deviation around a moving baseline.

```
# (continued from Standard Workflow)
# Adaptive subtractive normalization
preproc <- ts_norm_an(operation = "subtract")
set_example_seed()
preproc <- fit(preproc, ts)
tst <- transform(preproc, ts)

ts_head(tst, 3)
summary(tst[, 10])
```

Step by step

- `operation = "subtract"`: removes the adaptive level by **subtraction**
- Avoids numerical instability when the reference level is close to zero
- The transformed target behaves as a deviation around a moving baseline
- Prefer over `"divide"` for series with zero crossings

❏ **Key concept:** Additive adaptive detrending. Preserves absolute contrasts instead of relative ones. Reference: Ogasawara et al. (2010), IJCNN. doi:10.1109/IJCNN.2010.5596746

Adaptive Normalization Softdivide (`ts\_norm\_an` – softdivide)

The softdivide variant combines additive and divisive behavior through a stabilized denominator. Near zero, it acts like subtraction; at larger magnitudes, it acts like division. It is the **compromise operator** of the adaptive family — useful when the series alternates between regions close to zero and regions with heteroscedastic scale.

```
# (continued from Standard Workflow)
# Adaptive softdivide normalization
preproc <- ts_norm_an(operation = "softdivide",
                      scale = "sd", lambda = 1)
set_example_seed()
preproc <- fit(preproc, ts)
tst <- transform(preproc, ts)

ts_head(tst, 3)
summary(tst[, 10])
```

Step by step

- `operation = "softdivide"`: stabilized denominator blends addition and division
- Parameter `lambda`: controls the smoothness of the transition between the two regimes
- Near zero → additive behavior (like `"subtract"`)
- At larger magnitudes → divisive behavior (like `"divide"`)
- Reference: Huber (1964) — robust location estimation

📌 **Key concept:** Compromise operator between addition and division. Avoids instability without sacrificing scale invariance in higher-amplitude regions. Reference: Ogasawara et al. (2010), IJCNN; Huber (1964), Ann. Math. Stat.

Adaptive Normalization Asinh (`ts\_norm\_an` – asinh)

The asinh variant applies the **inverse hyperbolic sine** (arcsinh) transformation to the adaptive contrast. Near zero, it behaves like an additive difference; for larger deviations, it becomes progressively log-like — without an abrupt transition. It is a smooth bridge between additive and multiplicative interpretations.

```
# (continued from Standard Workflow)

# Adaptive normalization asinh
preproc <- ts_norm_an(operation = "asinh",
                      scale = "sd", lambda = 1)
set_example_seed()
preproc <- fit(preproc, ts)
tst <- transform(preproc, ts)

ts_head(tst, 3)
summary(tst[, 10])
```

Step by step

- `operation = "asinh"`: applies arcsinh to the adaptive contrast
- Approximately linear near zero — no divisive instability
- Progressively log-like for larger deviations
- Smooth and continuous transition — no explicit cutoff parameter
- References: Burbidge et al. (1988); Bellemare & Wichman (2020)

📌 **Key concept:** Smooth non-linear bridge between additive and log-multiplicative regimes. Ideal when the series has deviations of highly varying magnitudes. References: Burbidge et al. (1988), JASA; Bellemare & Wichman (2020), Oxford Bull. Econ. Stat.

Exponential Adaptive Normalization (ts_norm_ean)

Exponential Adaptive Normalization (EAN) computes **exponentially weighted** means and variances within each window. The parameter `nw` controls the effective span: smaller values adapt faster. It combines temporal adaptation with smoothing, making it ideal for series with non-stationary mean and variance.

```
# (continued from Standard Workflow)
# EAN with effective window nw=3
# (faster adaptation)
preproc <- ts_norm_ean(nw = 3)
preproc <- fit(preproc, ts)
tst <- transform(preproc, ts)

ts_head(tst, 3)
# t0: 0.4779, 0.4688, 0.4616

summary(tst[,10])
# Min. :0.4545 Max. :0.5437
# Mean :0.4911
```

Step by step

- Computes **exponentially weighted** means and variances within each window
- Parameter `nw`: effective span — smaller values adapt faster
- Centers and scales each window by the adaptive statistics
- Smooths transient noise while tracking recent changes
- Output with very low variance indicates strong scale compression

📌 **Key concept:** EAN combines temporal adaptation with exponential smoothing. Ideal for series with non-stationary mean/variance. Reference: Ogasawara et al. (2010), IJCNN.

Differentiation Normalization (ts_norm_diff)

Differentiation replaces each value with its **change** relative to the previous observation. This removes trend effects and stabilizes the series mean before scaling. The window loses one column after the transformation, and the model then predicts changes rather than absolute levels.

```
# (continued from Standard Workflow)
# Differentiation normalization
preproc <- ts_norm_diff()
preproc <- fit(preproc, ts)
tst <- transform(preproc, ts)
```

```
ts_head(tst, 3)
# t0: 0.2373, 0.1401, 0.0653
# (window reduces to 9 columns)
```

```
summary(tst[,9])
# Min. :0.00000 Max. :1.00000
# Median :0.29337 Mean :0.40975
```

Step by step

- Each value is replaced by its **change** relative to the previous observation
- The window loses one column (from 10 to 9) after differencing
- Reduces trend effects and stabilizes the series mean
- Facilitates subsequent scaling and modeling
- Interpretation changes: the model predicts **changes**, not absolute levels

📌 **Key concept:** Differencing stabilizes the mean before scaling. Reference: Box, Jenkins, Reinsel & Ljung (2015), *Time Series Analysis: Forecasting and Control*. Wiley.

Acknowledgements



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>

